`SLiMfast`'s user guide

Introduction

Short Linear Motifs (or SLiMs) are short functional motifs in protein sequences involved in several cellular processes and are usually found in disordered regions [1].

`SLiMfast` is a Python package implementing a user-friendly pipeline to check whether the Short Linear Motifs found (or predicted to be) in a specific protein sequence fall within structured or disordered regions.

`SLiMfast`'s main purpose is to weed out motif instances found within highly structured regions since they are unlikely to be functional. In this context, `SLiMfast` can be used as part of larger motif discovery frameworks and as a filtering tool to discriminate between potential SLiM candidates and false positive hits.

Installation

We provide detailed instructions on installing `SLiMfast` using a Python virtual environment or a `conda` environment in the `INSTALL.md` file.

Usage

The `SLiMfast` pipeline consists of four command-line tools (`slimfast_get_structures`, `slimfast_get_alphafold`, `slimfast_get_homologs`, and `slimfast_find_slims`) designed for the different steps of motif finding and filtering. They can be used one after the other to run the entire pipeline or separately if only part of it is needed.

A fifth command-line utility, `slimfast_run_pipeline`, can be used to run the pipeline (or only specific steps) for several UniProt IDs and the SLiMs to be found for each of them.

Several options determining the behavior of each tool are defined in a YAML configuration file passed as input. The file contains options relevant to each tool, so you only need one configuration file to run the entire pipeline.

`slimfast_get_structures`

The first executable in `SLiMfast`'s toolkit deals with gathering all available experimental structures for the protein of interest from the Protein Data Bank [2] and storing data about them into an easily-readable CSV file.

Command line

```
slimfast_get_structures [-h] [-id] UNIPROT_ID [-op OUTPUT_PROTEIN_FILE]
                        -cr CONFIGURATION_FILE [-lf LOG_FILE] [-ls]
                        [-v] [-vv]
```

Options

`$INSTALLDIR` is a placeholder for the path to the directory where you have installed `SLiMfast`.

Option	Description
`-h`, `--help`	Show the help message and exit.
`-id`, `-uniprot-id`	UniProt ID of the protein of interest.
`-op`, `--output-protein-file`	Name of the output CSV file containing the structures of the protein of interest found in the Protein Data Bank. The default is `structures.csv`.
`-cr`, `--configrun-file`	YAML configuration file for running any of the `SLiMfast` tools. If it is a name without extension, it is assumed to be a YAML file in `$INSTALLDIR/SLiMfast/SLiMfast/config_run` with that name.
`-lf`, `--log-file`	Name of the log file. The default is `slimfast_get_structures.log`.
`-ls`, `--log-to-stdout`	Log also to the standard output.
`-v`, `--verbose`	Verbose (INFO level) logging.
`-vv`, `--verbose-debug`	More verbose (DEBUG level) logging.

`slimfast_get_alphafold`

Since the release of the AlphaFold Protein Structure Database [3], researchers can now access high-quality AlphaFold2 predictions of the structures of most full-length proteins in several proteomes.

Therefore, `SLiMfast` provides the `slimfast_get_alphafold` tool to gather the model(s) available for a given protein of interest to use in the search for motifs.

Command line

```
slimfast_get_alphafold [-h] [-id] UNIPROT_ID [-oa OUTPUT_ALPHAFOLD_FILE]
                       -cr CONFIGURATION_FILE [-lf LOG_FILE] [-ls]
                       [-v] [-vv]
```

Options

`$INSTALLDIR` is a placeholder for the path to the directory where you have installed `SLiMfast`.

Option	Description
`-h`, `--help`	Show the help message and exit.
`-id`, `-uniprot-id`	UniProt ID of the protein of interest.
`-oa`, `--output-alphafold-file`	Name of the output CSV file containing the structures of the protein of interest found in the AlphaFold Protein Structure Database. The default is `alphafold.csv`.
`-cr`, `--configrun-file`	YAML configuration file for running any of the `SLiMfast` tools. If it is a name without extension, it is assumed to be a YAML file in `$INSTALLDIR/SLiMfast/SLiMfast/config_run` with that name.
`-lf`, `--log-file`	Name of the log file. The default is `slimfast_get_alphafold.log`.
`-ls`, `--log-to-stdout`	Log also to the standard output.
`-v`, `--verbose`	Verbose (INFO level) logging.
`-vv`, `--verbose-debug`	More verbose (DEBUG level) logging.

`slimfast_get_homologs`

Despite the accuracy of the AlphaFold2 models, you may still want to inspect only experimental structures. However, no structures may be available for your protein of interest in the Protein Data Bank. In these cases, you can use `slimfast_get_homologs` to obtain a list of structures of proteins homologous to the one of interest available in the Protein Data Bank.

`slimfast_get_homologs` takes advantage of the HHpred pipeline [4] to search for homologous proteins (using the `HHblits` tool) and then to query the Protein Data Bank for structures of the homologs (using `HHsearch`).

Command line

```
slimfast_get_homologs [-h] [-id] UNIPROT_ID [-oh OUTPUT_HOMOLOGS_FILE]
                      [-oal OUTPUT_ALIGNMENTS_FILE] -cr CONFIGURATION_FILE
                      [-lf LOG_FILE] [-ls] [-v] [-vv]
```

Options

`$INSTALLDIR` is a placeholder for the path to the directory where you have installed `SLiMfast`.

Option	Description
`-h`, `--help`	Show the help message and exit.

Option	Description
`-id`, `-uniprot-id`	UniProt ID of the protein of interest.
`-oh`, `--output-homologs-file`	Name of the output CSV file containing the structures of the homologs of the protein of interest found in the Protein Data Bank. The default is `homologs.csv`.
`-oal`, `--output-alignments-file`	Name of the output JSON file containing the alignments between each of the structures of the homologs of the protein of interest found in the Protein Data Bank and the protein itself. The default is `alignments.json`.
`-cr`, `--configrun-file`	YAML configuration file for running any of the `SLiMfast` tools. If it is a name without extension, it is assumed to be a YAML file in `$INSTALLDIR/SLiMfast/SLiMfast/config_run` with that name.
`-lf`, `--log-file`	Name of the log file. The default is `slimfast_get_homologs.log`.
`-ls`, `--log-to-stdout`	Log also to the standard output.
`-v`, `--verbose`	Verbose (INFO level) logging.
`-vv`, `--verbose-debug`	More verbose (DEBUG level) logging.

`slimfast_find_slims`

The fourth utility in the `SLiMfast` kit is needed to look for the SLiMs of interest in the structures obtained through `slimfast_get_structures`, `slimfast_get_alphafold`, and `slimfast_get_homologs` and assess whether they fall into structured or disordered regions using the `DSSP` program [5].

The outputs from `slimfast_get_structures`, `slimfast_get_alphafold`, and `slimfast_get_homologs` can be passed to `slimfast_find_slims`, but neither of them is strictly needed. For instance, you can pass only the CSV file outputted by `slimfast_get_structures` and the CSV file and JSON file generated by `slimfast_get_homologs` or any other combination of the outputs.

The only two outputs that always need to be passed together are the CSV file containing the structures of the homologs, and the corresponding JSON file since both the list of structures (and associated metadata) and the alignments of the structures' sequences to the protein are needed to check for the SLiMs of interest.

Indeed, `slimfast_find_slims` checks for a SLiM in a homolog by checking if the SLiM is found in the portion of the homologous sequence aligned to the protein of interest. In other words, `slimfast_find_slims` checks if the SLiM is perfectly conserved in the homologous sequence.

Since one of the ideas that prompted `SLiMfast`'s development was to find structures encompassing the SLiMs of interest that could be then used as a template to model the SLiM, `slimfast_find_slims` only looks for perfect matches between the SLiM and its corresponding region in the homologous sequence.

Command line

```
slimfast_find_slims [-h] -id UNIPROT_ID [-ip INPUT_PROTEIN_FILE]
                    [-ia INPUT_ALPHAFOLD_FILE] [-ih INPUT_HOMOLOGS_FILE]
                    [-ial INPUT_ALIGNMENTS_FILE] -is INPUT_SLIMS_FILE
                    [-osp OUTPUT_SLIMS_PROTEIN_FILE]
                    [-osa OUTPUT_SLIMS_ALPHAFOLD_FILE]
                    [-osh OUTPUT_SLIMS_HOMOLOGS_FILE]
                    [-osd OUTPUT_SLIMS_DISPROT_FILE]
                    -cr CONFIGURATION_FILE [-lf LOG_FILE]
                    [-ls] [-v] [-vv]
```

Options

`$INSTALLDIR` is a placeholder for the path to the directory where you have installed `SLiMfast`.

Option	Description
`-h`, `--help`	Show the help message and exit.
`-id`, `-uniprot-id`	UniProt ID of the protein of interest.
`-ip`, `--input-protein-file`	Input CSV file containing the structures of the protein of interest found in the Protein Data Bank.
`-ia`, `--input-alphafold-file`	Input CSV file containing the structures of the protein of interest found in the AlphaFold Protein Structure Database.
`-ih`, `--input-homologs-file`	Input CSV file containing the structures of the homologs of the protein of interest found in the Protein Data Bank.
`-ial`, `--input-alignments-file`	Input JSON file containing the alignments between each of the structures of the homologs found in the Protein Data Bank and the protein of interest.
`-is`, `--input-slims-file`	Input CSV file containing the SLiMs in the UniProt sequence of the protein of interest.
`-osp`, `--output-slims-protein-file`	Name of the output CSV file containing the SLiMs found in the structures of the protein of interest found in the Protein Data Bank. The default is `slims_structures.csv`.
`-osa`, `--output-slims-alphafold-file`	Name of the output CSV file containing the SLiMs found in the structures of the protein of interest found in the AlphaFold Protein Structure Database. The default is `slims_alphafold.csv`.
`-osh`, `--output-slims-homologs-file`	Name of the output CSV file containing the SLiMs found in the structures of the homologs of the protein of interest found in the Protein Data Bank. The default is `slims_homologs.csv`.

Option	Description
`-osd`, `--output-slims-disprot-file`	Name of the output CSV file containing the SLiMs not found in the structures available either for the protein of interest or for the homologs, but annotated as disordered in `DisProt`. The default is `slims_disprot.csv`.
`-cr`, `--configrun-file`	YAML configuration file for running any of the `SLiMfast` tools. If it is a name without extension, it is assumed to be a YAML file in `$INSTALLDIR/SLiMfast/SLiMfast/config_run` with that name.
`-lf`, `--log-file`	Name of the log file. The default is `slimfast_get_homologs.log`.
`-ls`, `--log-to-stdout`	Log also to the standard output.
`-v`, `--verbose`	Verbose (INFO level) logging.
`-vv`, `--verbose-debug`	More verbose (DEBUG level) logging.

`slimfast_run_pipeline`

The last `SLiMfast` tool is `slimfast_run_pipeline`, which provides a convenient way to run the steps of the `SLiMfast` pipeline automatically for several UniProt IDs and the associated SLiMs of interest.

Furthermore, on multi-core architectures, `SLiMfast` is able to parallelize the workload efficiently onto as many cores as you want and, therefore, speed up the calculation.

Command line

```
slimfast_run_pipeline [-h] [-dgs] [-dga] [-dgh] [-dfs]
                      -ids INPUT_IDS_SLIMS_FILE
                      [-op OUTPUT_PROTEIN_FILE]
                      [-oa OUTPUT_ALPHAFOLD_FILE]
                      [-oh OUTPUT_HOMOLOGS_FILE]
                      [-oal OUTPUT_ALIGNMENTS_FILE]
                      [-osp OUTPUT_SLIMS_PROTEIN_FILE]
                      [-osa OUTPUT_SLIMS_ALPHAFOLD_FILE]
                      [-osh OUTPUT_SLIMS_HOMOLOGS_FILE]
                      [-osd OUTPUT_SLIMS_DISPROT_FILE]
                      -cr CONFIGRUN_FILE [-lf LOG_FILE]
                      [-ls] [-v] [-vv] [-n NPROC]
```

Options

Option	Description
`-h`, `--help`	Show the help message and exit.

Option	Description
`-dgs`, `--do-get-structures`	Run `slimfast_get_structures`.
`-dga`, `--do-get-alphafold`	Run `slimfast_get_alphafold`.
`-dgh`, `--do-get-homologs`	Run `slimfast_get_homologs`.
`-dfs`, `--do-find-slims`	Run `slimfast_find_slims`.
`-ids`, `--input-ids-slims-file`	Input CSV file containing the UniProt IDs of the proteins of interest and the SLiMs to be found in their sequences.
`-op`, `--output-protein-file`	Name of the output CSV file containing the structures of the protein of interest found in the Protein Data Bank. The default is `structures.csv`.
`-oa`, `--output-alphafold-file`	Name of the output CSV file containing the structures of the protein of interest found in the AlphaFold Protein Structure Database. The default is `alphafold.csv`.
`-oh`, `--output-homologs-file`	Name of the output CSV file containing the structures of the homologs of the protein of interest found in the Protein Data Bank. The default is `homologs.csv`.
`-oal`, `--output-alignments-file`	Name of the output JSON file containing the alignments between each of the structures of the homologs found in the Protein Data Bank and the protein of interest.
`-osp`, `--output-slims-protein-file`	Name of the output CSV file containing the SLiMs found in the structures of the protein of interest found in the Protein Data Bank. The default is `slims_structures.csv`.
`-osa`, `--output-slims-alphafold-file`	Name of the output CSV file containing the SLiMs found in the structures of the protein of interest found in the AlphaFold Protein Structure Database. The default is `slims_alphafold.csv`.
`-osh`, `--output-slims-homologs-file`	Name of the output CSV file containing the SLiMs found in the structures of the homologs of the protein of interest found in the Protein Data Bank. The default is `slims_homologs.csv`.
`-osd`, `--output-slims-disprot-file`	Name of the output CSV file containing the SLiMs not found in the structures available either for the protein of interest or for the homologs, but annotated as disordered in `DisProt`. The default is `slims_disprot.csv`.

Option	Description
`-cr`, `--configrun-file`	YAML configuration file for running any of the `SLiMfast` tools. If it is a name without extension, it is assumed to be a YAML file in `$INSTALLDIR/SLiMfast/SLiMfast/config_run` with that name.
`-lf`, `--log-file`	Name of the log file. The default is `slimfast_get_homologs.log`.
`-ls`, `--log-to-stdout`	Log also to the standard output.
`-v`, `--verbose`	Verbose (INFO level) logging.
`-vv`, `--verbose-debug`	More verbose (DEBUG level) logging.
`-n`, `--nproc`	Number of processes (cores) to use when running. The default is 1 process(es).

Example 1 - Running step by step for a single protein

Assume we want to run the entire `SLiMfast` pipeline for the human protein Bcl-2, whose UniProt ID is P10415.

First, we can create a directory named after Bcl-2's UniProt ID, so that all results pertinent to this protein will be stored in a dedicated folder. Then, we can enter the directory since all the pipeline's work will be carried out there.

```
mkdir P10415
cd P10415
```

Step 1 - `slimfast_get_structures`

Running

As a first step, we want to use `slimfast_get_structures` to get all the structures of human Bcl-2 available in the Protein Data Bank.

To do so in a reproducible way, we can create a Bash script, which we name `1-get_structures.sh`, where all the options are defined and the executable is launched. We are going to use the default `run.yaml` file provided with the package after having copied it into our directory.

```
# UniProt ID
id=P10415

# Configuration file
cr=run.yaml
```

```
# Name of the output CSV file
op=${id}_structures.csv

# Name of the log file
lf=${id}_structures.log

# Run with verbose logging, and log both to the log file and the standard output
slimfast_get_structures -id $id -cr $cr -op $op -lf $lf -v -ls
```

Since we decided to log both to the log file and to the standard output, we will see the log messages printed to the console as well (or wherever the standard output is redirected).

Output

If the run was completed successfully, we should see these new files in our directory `P10415`:

- `P10415_structures.csv`, which contains the data frame with the data about the structures of the protein of interest found in the Protein Data Bank.
- `P10415_structures.csv`, which contains the log messages.

`P10415_structures.csv` reports each unique chain found representing the protein of interest. The first lines of the file will look similar to this:

```
struct_id,struct_chain,struct_segments,uniprot_id,uniprot_segments,method,resolu
tion,deposition_date
6GL8,A,9-33;92-92;93-98;103-None;P10415,9-33;66-66;93-108;103-206,X-RAY
DIFFRACTION,1.4,2018-05-23
4LXD,A,1-34;51-166,P10415,1-34;92-207,X-RAY DIFFRACTION,1.9,2013-07-29
202F,A,6-48;89-201,P10415,3-34;50-66;93-207,SOLUTION NMR,,2006-11-29
```

The data frame has eight columns:

- `struct_id`, which contains the ID of the structure (in this case, the PDB ID).
- `struct_chain`, which contains the unique chain represented by the entry.
- `struct_segments`, which contains the segments of the UniProt sequence corresponding to the protein of interest covered by the structure, reported with the numbering used in the structure (which may differ from the UniProt numbering).
- `uniprot_id`, which contains the UniProt ID of the protein represented by the entry (structure and specific chain).
- `uniprot_segments`, which contains the segments of the UniProt sequence corresponding to the protein of interest covered by the structure, reported with the UniProt numbering.
- `method`, which contains the method used to solve the protein structure.
- `resolution`, which contains the structure's resolution (if applicable). This field is left empty for structures coming from methods such as solution NMR.
- `deposition_date`, which contains the date when the structure was deposited into the database.

Sometimes, data regarding the segments of the UniProt sequence covered by the structures are missing (exemplified by the `None` values in the `struct_segments` field). `SLiMfast` takes these annotations from the Protein Data Bank in Europe (PDBe) [6]. It does not happen often but can occur if the structure is, for example, very new.

Since structures reporting these data as missing are not considered while looking for SLiMs (since we are unsure of which portions of the sequence are actually covered), we highly recommend manually annotating these fields in the CSV file so that the corresponding structures can be used for the motif search.

Step 2 - `slimfast_get_alphafold`

Running

To gather the models of our protein of interest generated by AlphaFold2, we can wrap the call to `slimfast_get_alphafold` into the `2-get_alphafold.sh` script.

```
# UniProt ID
id=P10415

# Configuration file
cr=run.yaml

# Name of the output CSV file
oa=${id}_alpahfold.csv

# Name of the log file
lf=${id}_alphafold.log

# Run with verbose logging, and log both to the log file and the standard output
slimfast_get_alphafold -id $id -cr $cr -oa $op -lf $lf -v -ls
```

Output

If the run was successful, you will have these output files in your directory:

- `P10415_alphafold.csv`, which contains the data frame with the info about the structures of the protein of interest found in the AlphaFold Protein Structure Database.
- `P10415_alphafold.csv`, which contains the log messages.

`P10415_alphafold.csv` reports each unique structure found in the database representing the protein of interest. The first lines of the file will look similar to this:

```
struct_id,struct_chain,struct_segments,struct_version,struct_fragment,uniprot_id
,uniprot_segments,uniprot_isoform,alphafold_id,deposition_date
AF-P10415-F1-model_v3,A,1-1,3,1,P10415,1-239,,AF-P10415-F1,2022-06-01
AF-P10415-F1-model_v3,A,1-1,3,1,P10415,1-239,,AF-P10415-F1,2022-06-01
```

The data frame has twelve columns:

- `struct_id`, which contains the ID of the structure (in this case, the complete name of the structure in the AlphaFold Protein Structure Database).
- `struct_chain`, which contains the unique chain represented by the entry.
- `struct_segments`, which contains the segments of the UniProt sequence corresponding to the protein of interest covered by the structure, reported with the numbering used in the structure (which may differ from the UniProt numbering).
- `struct_version`, which contains the version of the AlphaFold2 pipeline used to generate the structure.

- `struct_segment`, which contains the segment identifier for the portion of the UniProt sequence covered by the structure. This will be `1` for most structures since structures of proteins up to 2700 residues in length are predicted as single structures by AlphaFold2. However, bigger proteins are split into multiple segments. Therefore, this integer may vary according to the protein segment the structure covers.
- `uniprot_id`, which contains the UniProt ID of the protein represented by the entry (structure and specific chain).
- `uniprot_segments`, which contains the segments of the UniProt sequence corresponding to the protein of interest covered by the structure, reported with the UniProt numbering.
- `uniprot_isoform`, which contains the isoform of the protein the structure refers to. If not specified, it should be the most common isoform.
- `alphafold_id`, which contains the ID of the structure in the AlphaFold Protein Structure Database. Please note that more than one structure can share this ID since structures representing different segments of the same protein and structures of the same protein (and corresponding fragments) generated with different versions of the AlphaFold2 pipeline will have the same ID.
- `deposition_date`, which contains the date when the structure was deposited into the database.

Step 3 - `slimfast_get_homologs`

Running

As done for `slimfast_get_structures` and `slimfast_get_alphafold`, we can encapsulate the command line to launch `slimfast_get_homologs` into a Bash script (`3-slimfast_get_homologs.sh`).

```
# UniProt ID
id=P10415

# Configuration file
cr=run.yaml

# Name of the output CSV file
oh=${id}_homologs.csv

# Name of the output JSON file
oal=${id}_homologs.json

# Name of the log file
lf=${id}_homologs.log

# Run with verbose logging, and log both to the log file and the standard output
slimfast_get_homologs -id $id -cr $cr -oh $oh -oal $oal -lf $lf -v -ls
```

Output

Upon the run's completion, we will have these new items in the working directory:

- A `hhpred` folder, which contains the outputs from `HHblits` and `HHsearch`, the two software composing the `HHpred` pipeline:
    - `hhblits.log` is the output captured from the `HHblits` run.

    - `hhsearch.log` is the output captured from the `HHsearch` run.
    - `P10415.a3m` is the output of `HHblits` containing the homologs found.
    - `P10415.hhsearch` is the output of `HHsearch` containing the homologs whose structures were found in the Protein Data Bank.

  According to the options given to `HHblits` and `HHsearch` in the configuration file, you may also have additional output files.

- `P10415_homologs.csv`, which contains the data frame with the data about the structures of the homologs found in the Protein Data Bank.
- `P10415_homologs.json`, which contains the alignments between the structures of the homologs found in the Protein Data Bank and the protein of interest.
- `P10415_homologs.log`, which contains the log messages.

An example of a data frame produced by `slimfast_get_homologs` would be:

```
hit_id,num,hit,description,prob,e_value,p_value,score,ss,cols,uniprot_segments,s
truct_segments,struct_id,struct_chain,identities,similarity,sum_probs,template_n
eff,organism,uniprot_coverage_percent,method,resolution,deposition_date
1_5jsn.A,1,5jsn.A,Apoptosis regulator Bcl,100.0,1.6E-40,1.8E-45,284.2,0.0,206,1-
206,1-206,5jsn,A,100.0,1.602,139.4,6.7,Homo sapiens,86.2,X-RAY
DIFFRACTION,2.1,2016-05-09
```

The data frame has twenty-three columns:

- `hit_id`, which contains the unique ID of the `HHsearch` hit corresponding to the structure.
- `num`, which contains the hit number.
- `hit`, which contains the hit name (PDB ID plus chain).
- `description`, which contains a short description of the protein corresponding to the hit.
- `e_value`, which contains the E-value associated with the hit.
- `p_value`, which contains the P-value associated with the hit.
- `score`, which contains the score assigned by `HHsearch` to the hit.
- `ss`, which contains the secondary structure score of a hit. it comes from the alignment of secondary structure assignments between query (the UniProt sequence of the protein of interest) and template (the hit), as provided by PSIPRED (3-state) or determined by DSSP (8-state) [7].
- `cols`, which contains the number of aligned residues (named by `HHsearch` "columns" or "cols") between the hit and the UniProt sequence of the protein of interest.
- `uniprot_segments`, which contains the segments of the UniProt sequence corresponding to the protein of interest covered by the structure, reported with the UniProt numbering.
- `struct_segments`, which contains the segments of the UniProt sequence corresponding to the protein of interest covered by the structure, reported with the numbering used in the structure (which may differ from the UniProt numbering).
- `struct_id`, which contains the ID of the structure (in this case, the PDB ID).
- `struct_chain`, which contains the unique chain represented by the entry.
- `identities`, which contains the identity percentage between the sequence represented by the hit and the UniProt sequence of the protein of interest.
- `similarity`, which contains the percentage of similarity between the sequence represented by the hit and the UniProt sequence of the protein of interest.
- `method`, which contains the method used to solve the protein structure.

- `resolution`, which contains the structure's resolution (if applicable). This field is left empty for structures coming from methods such as solution NMR.
- `sum_probs`, which contains the expected number of correctly aligned residues calculated with the MAC algorithm [8].
- `template_neff`, which contains the NEFF distribution of the templates (hits) found by `HHsearch` [7].
- `organism`, which contains the scientific name of the organism from which the protein sequence represented by the hit comes.
- `uniprot_coverage_percent`, which represents the percentage of the UniProt sequence of the protein of interest covered by the hit.
- `deposition_date`, which contains the date when the structure was deposited into the database.
- `deposition_date`, which contains the date when the structure was deposited into the database.

Step 4 - `slimfast_find_slims`

We can now run `slimfast_find_slims` using the output files produced by Step 1, Step 2, and Step 3.

We just need an additional input file, namely the list of SLiMs we want to look for in the structures. This is a CSV file like the following:

```
BH3,92,VVHLTLRQAGDDFSRR,107
```

The file has four columns:

- The first column contains the name of the motif. It can be any arbitrary name chosen by the user to identify the motif. Since this name is not assumed to be unique, it can also indicate the class the motif belongs to. For instance, in this case, since we chose the BH3 motif of Bcl-2 we named our SLiM `BH3` in the input file.
- The second column contains the start of the motif in the UniProt sequence of the protein.
- The third column contains the sequence of the motif in the UniProt sequence of the protein.
- The fourth column contains the end of the motif in the UniProt sequence of the protein.

Before running `slimfast_find_slims`, remember that the structures of the protein of interest found in the Protein Data Bank are prioritized with respect to the ones found in the AlphaFold Protein Structure Database, and the latter are prioritized with respect to the structures of the homologous proteins.

Furthermore, remember that how you sorted the structures in each input file (using the options provided in the configuration file or manually) affects how the SLiMs will be searched. Indeed, `slimfast_find_slims` assumes that the "best" structures appear at the beginning of each input file. Therefore, if all SLiMs are found using only a subset of the structures in the file, the remaining ones will not be considered.

Having all the input files in place, we can now devise a Bash script to wrap our `slimfast_find_slims` run.

```
# UniProt ID
id=P10415
```

```
# Configuration file
cr=run.yaml

# Input file containing the SLiMs to be found
is=slims.txt

# Input CSV file containing the structures of the protein of interest
# found in the Protein Data Bank
ip=${id}_structures.csv

# Input CSV file containing the structures of the protein of interest
# found in the AlphaFold Protein Structure Database
ia=${id}_alphafold.csv

# Input CSV file containing the structures of the homologs of the
# protein of interest found in the Protein Data Bank
ih=${id}_homologs.csv

# Input JSON file containing the alignments between each of the
# structures of the homologs found in the Protein Data Bank and
# the protein of interest
ial=${id}_homologs.json

# Log file
lf={id}_slims.log

# Run
slimfast_find_slims -id $id -cr $cr -is $is -ip $ip -ia $ia -ih $ih -ial $ial -
lf $lf -v -ls
```

Since we have only one motif to be found, `slimfast_find_slims` will produce, at mos,t one output file, according to where the SLiM was found. For example, if the SLiM was found in a structure of Bcl-2 found in the Protein Data Bank (i.e., coming from the `P10415_structures.csv` input file), we will have the following CSV file named `slims_structures.csv`:

```
source,struct_id,struct_chain,slim_name,slim_start,slim_sequence,slim_end,disord
ered_region
structures,4LXD,A,BH3,92,VVHLTLRQAGDDFSRR,107,False
```

The data frame has eight columns:

- `struct_id`, which contains the ID of the structure.
- `struct_chain`, which contains the unique chain represented by the entry.
- `slim_name`, which contains the name assigned to the SLiM by the user in the input CSV file.
- `slim_start`, which contains the start of the motif in the UniProt sequence of the protein.
- `slim_sequence`, which contains the sequence of the motif in the UniProt sequence of the protein.
- `slim_end`, which contains the end of the motif in the UniProt sequence of the protein.
- `disordered_region`, which contains whether the SLiM was found in a completely disordered region in the structure, as assessed by DSSP.

Since the SLiM was found already in the structures of the protein available in the Protein Data Bank, those produced by AlphaFold2 or those of the homologs were not even considered during the search, and no output was produced for them.

If the SLiM had not been found in the protein structures found in the Protein Data Bank, `slimfast_find_slims` would have tried to find it in the structures gathered from the AlphaFold Protein Structure Database (and stored in the `Q10415_alphafold.csv` output file). Upon finding it, `slimfast_find_slims` would have written a `slims_alphafold.csv` CSV file containing a data frame with an identical structure to the one described above, with `alphafold` instead of `structures` written in the `source` field and details about the structure and the SLiM in the others.

If the SLiM were not found in the AlphaFold2 structures either, `slimfast_find_slims` would have tried to find it in the structure of the homologs (in the `Q10415_homologs.csv` file) and, if successful, would have produced a `slims_homologs.csv` CSV file containing a data frame with the same structure described above, with `homologs` in the `source` field.

Finally, if none of the structures provided contained the SLiM, `slimfast_find_slims` would have looked for annotations of the protein region containing the SLiM in the DisProt database [9] to assess whether the region had been classified as structured or disordered. In this case, if any annotations had been found, a `slims_disprot.csv` file would have been produced, with a data frame containing the `disprot` string in the `source` field and no `struct_id` or `struct_chain` field, but with all the other fields describing the SLiM.

Example 2 - Running in one go for multiple proteins

Suppose that we have two proteins of the Bcl-2 family, Bcl-2 (UniProt ID: P10415) and Bcl-xL (UniProt ID: , for which we want to find whether their BH3 and BH2 motifs fall within a structured or disordered region, considering all possible available structures for both proteins. We can do that for both proteins in one go using `slimfast_run_pipeline`.

One step - `slimfast_run_pipeline`

Input

First of all, we need a CSV file containing both the UniProt IDs of the proteins and the SLiMs we want to search for each of them.

Therefore, we create the `ids_and_slims.csv` file:

```
uniprot_id,slim_type,slim_start,slim_sequence,slim_end
P10415,BH2,187,TWIQDNGGWDAFVELY,202
P10415,BH3,92,VVHLTLRQAGDDFSRR,107
Q07817,BH2,180,PWIQENGGWDTFVELY,195
Q07817,BH3,86,VKQALREAGDEFELR,100
```

Running

Then, we create a Bash script to launch `slimfast_run_pipeline`. We leave the default names for all the output files and the log file.

```
# Configuration file
cr=run.yaml

# Input file containing the UniProt IDs and the SLiMs to be found
ids=ids_and_slims.txt

# Number of processes (cores) to be used
n=2

# Run using verbose logging. For each protein, run:
# - slimfast_get_structures
# - slimfast_get_alphafold
# - slimfast_get_homologs
# - slimfast_find_slims
slimfast_run_pipeline --do-get-structures --do-get-alphafold --do-get-homologs --do-find-slims -ids $ids -cr $cr -v -n $n
```

Output

At the end of the run, we should find two directories named `P10415` and `Q07817` in our working directory, each containing the results of `slimfast_get_structures`, `slimfast_get_alphafold`, `slimfast_get_homologs`, and `slimfast_find_slims`, together with their log files.

All output and log files will have the name given with the corresponding option (or the default name, if none was given) prefixed with the UniProt ID of the protein they refer to.

A log file named `slimfast_run_pipeline.log` will also appear in the working directory, containing general information about the progress of the pipeline.

More detailed logging about each run is contained in the corresponding log files in each subdirectory (i.e., `P10415_slimfast_get_structures.log` in the `P10415` subdirectory for the `slimfast_get_structures` run performed on Bcl-2).

References

1. Davey, Norman E., et al. "Attributes of short linear motifs." Molecular BioSystems* 8.1 (2012): 268-281. ↩

2. Berman, Helen M., et al. "The protein data bank." *Nucleic acids research* 28.1 (2000): 235-242. ↩

3. Varadi, Mihaly, et al. "AlphaFold Protein Structure Database: massively expanding the structural coverage of protein-sequence space with high-accuracy models." *Nucleic acids research* 50.D1 (2022): D439-D444. ↩

4. Söding, Johannes, Andreas Biegert, and Andrei N. Lupas. "The HHpred interactive server for protein homology detection and structure prediction." *Nucleic acids research* 33.suppl_2 (2005): W244-W248. ↩

5. Kabsch, Wolfgang, and Christian Sander. "Dictionary of protein secondary structure: pattern recognition of hydrogen-bonded and geometrical features." *Biopolymers: Original Research on Biomolecules* 22.12 (1983): 2577-2637. ↩

6. Velankar, Sameer, et al. "PDBe: protein data bank in Europe." *Nucleic acids research* 39.suppl_1 (2010): D402-D410. ↩

7. Gabler, Felix, et al. "Protein sequence analysis using the MPI bioinformatics toolkit." *Current Protocols in Bioinformatics* 72.1 (2020): e108. ↩ ↩

8. Hildebrand, Andrea, et al. "Fast and accurate automatic structure prediction with HHpred." *Proteins: Structure, Function, and Bioinformatics* 77.S9 (2009): 128-132. ↩

9. Sickmeier, Megan, et al. "DisProt: the database of disordered proteins." *Nucleic acids research* 35.suppl_1 (2007): D786-D793. ↩